

MediTrack

Hospital Data Analysis — Full-Stack Edition

Complete Project Documentation

MySQL 8.0+	Flask REST API	React 19 + Vite	Recharts
8 Tables	26 Endpoints	8 Pages	Live Dashboard

Tech Stack: MySQL · Flask · React · Vite · Recharts · Axios

Database: hospital_db | Backend: localhost:5000 | Frontend: localhost:3000

This document contains the complete database schema, backend source code, frontend source code, and live screenshots of every page in the running application.

Table of Contents

1. Project Overview
2. System Architecture
3. Database Schema (MySQL)
4. Backend — Flask REST API
5. Frontend — React Application
6. Live Application — Page-by-Page Walkthrough
7. API Endpoint Reference
8. Setup & Run Instructions
9. Summary

1. Project Overview

MediTrack is a full-stack hospital management and analytics application built on top of a MySQL relational database. It extends an original SQL-only data-analysis project into a complete, runnable web application with a REST API backend and an interactive React frontend.

What it does

- Stores and manages hospital data across 8 normalized tables — patients, doctors, departments, admissions, billing, medications, lab tests, and staff
- Exposes full CRUD (Create, Read, Update, Delete) operations for every entity through a Flask REST API
- Provides a live analytics dashboard with 6 interactive charts powered by real SQL aggregate queries — including window functions
- Lets the user add, edit, search, filter, and delete records directly from the browser, with changes persisted instantly to MySQL

Why it was built this way

The original project was a pure SQL portfolio piece — schema, seed data, and 40+ analysis queries run manually in MySQL Workbench. This full-stack version turns those same queries into live, clickable software: every KPI on the dashboard is the direct output of a SQL query running against the database in real time, not a static screenshot.

2. System Architecture

Layer	Technology	Responsibility
Database	MySQL 8.0+	Stores all hospital data — patients, admissions, billing, etc.
Backend	Flask (Python)	REST API — 7 CRUD modules + 1 analytics module
Frontend	React 19 + Vite	Single-page app — dashboard + 7 CRUD pages
Charts	Recharts	Live bar / line / pie charts on the dashboard
HTTP Client	Axios	Frontend → Backend API calls

Request flow

When the browser loads the Dashboard page, React calls the Flask API at `localhost:5000/api/analytics/dashboard-summary` (and several other analytics endpoints).

Flask runs the corresponding SQL query against `hospital_db` using a pooled MySQL connection, returns the result as JSON, and React renders it into KPI cards and Recharts visualizations. CRUD pages (Patients, Doctors, etc.) follow the same pattern but call the resource-specific endpoints (GET / POST / PUT / DELETE).

3. Database Schema (MySQL)

The database consists of 8 tables with foreign key relationships, ENUM types for controlled categorical data, and a summary VIEW that powers the live dashboard.

Table	Description	Key Columns
departments	Hospital departments	department_id, name, head_doctor_id
doctors	Doctor profiles	doctor_id, specialization, experience_yrs
patients	Patient demographics	patient_id, date_of_birth, gender, blood_group
admissions	Admission records	admission_id, diagnosis, severity, status
billing	Financial transactions	total_amount, insurance_covered, payment_status
medications	Prescriptions per admission	drug_name, dosage, frequency
lab_tests	Diagnostic test results	test_name, result_status, cost
staff	Support staff records	role, shift, salary

Entity Relationships

departments → doctors → admissions → billing / medications / lab_tests
patients → admissions departments → staff

Full Schema DDL

```
schema/01_schema.sql

-- =====
-- Hospital Data Analysis Project - Database Schema
-- File: 01_schema.sql | Dialect: MySQL 8.0+
-- =====

CREATE DATABASE IF NOT EXISTS hospital_db;
USE hospital_db;

CREATE TABLE IF NOT EXISTS departments (
  department_id INT AUTO_INCREMENT PRIMARY KEY,
  department_name VARCHAR(100) NOT NULL,
  location VARCHAR(100),
  head_doctor_id INT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS doctors (
  doctor_id INT AUTO_INCREMENT PRIMARY KEY,
  first_name VARCHAR(50) NOT NULL,
  last_name VARCHAR(50) NOT NULL,
  specialization VARCHAR(100) NOT NULL,
  department_id INT,
  experience_yrs INT,
  email VARCHAR(100) UNIQUE,
  phone VARCHAR(20),
```

```

joining_date DATE,
CONSTRAINT fk_doctor_dept FOREIGN KEY (department_id)
REFERENCES departments(department_id)
);

ALTER TABLE departments
ADD CONSTRAINT fk_dept_head FOREIGN KEY (head_doctor_id)
REFERENCES doctors(doctor_id);

CREATE TABLE IF NOT EXISTS patients (
patient_id INT AUTO_INCREMENT PRIMARY KEY,
first_name VARCHAR(50) NOT NULL,
last_name VARCHAR(50) NOT NULL,
date_of_birth DATE NOT NULL,
gender ENUM('Male','Female','Other') NOT NULL,
blood_group VARCHAR(5),
phone VARCHAR(20),
email VARCHAR(100),
address TEXT,
registered_on DATE DEFAULT (CURRENT_DATE)
);

CREATE TABLE IF NOT EXISTS admissions (
admission_id INT AUTO_INCREMENT PRIMARY KEY,
patient_id INT NOT NULL,
doctor_id INT NOT NULL,
department_id INT NOT NULL,
admission_date DATE NOT NULL,
discharge_date DATE,
diagnosis VARCHAR(255) NOT NULL,
severity ENUM('Mild','Moderate','Severe','Critical') NOT NULL,
admission_type ENUM('Emergency','Planned','Referral') NOT NULL,
room_number VARCHAR(10),
status ENUM('Admitted','Discharged','ICU','Transferred') DEFAULT 'Admitted',
CONSTRAINT fk_adm_patient FOREIGN KEY (patient_id) REFERENCES patients(patient_id),
CONSTRAINT fk_adm_doctor FOREIGN KEY (doctor_id) REFERENCES doctors(doctor_id),
CONSTRAINT fk_adm_dept FOREIGN KEY (department_id) REFERENCES departments(department_id)
);

CREATE TABLE IF NOT EXISTS billing (
bill_id INT AUTO_INCREMENT PRIMARY KEY,
admission_id INT NOT NULL UNIQUE,
patient_id INT NOT NULL,
total_amount DECIMAL(12,2) NOT NULL,
medication_cost DECIMAL(12,2) DEFAULT 0,
surgery_cost DECIMAL(12,2) DEFAULT 0,
consultation_fee DECIMAL(12,2) DEFAULT 0,
room_charges DECIMAL(12,2) DEFAULT 0,
insurance_covered DECIMAL(12,2) DEFAULT 0,
amount_paid DECIMAL(12,2) DEFAULT 0,
payment_status ENUM('Paid','Pending','Partial','Waived') DEFAULT 'Pending',
payment_method ENUM('Cash','Card','Insurance','UPI','Bank Transfer'),
bill_date DATE,
CONSTRAINT fk_bill_admission FOREIGN KEY (admission_id) REFERENCES admissions(admission_id),
CONSTRAINT fk_bill_patient FOREIGN KEY (patient_id) REFERENCES patients(patient_id)
);

CREATE TABLE IF NOT EXISTS medications (
medication_id INT AUTO_INCREMENT PRIMARY KEY,
admission_id INT NOT NULL,
drug_name VARCHAR(150) NOT NULL,
dosage VARCHAR(50),
frequency VARCHAR(50),
start_date DATE,
end_date DATE,
prescribed_by INT,
CONSTRAINT fk_med_admission FOREIGN KEY (admission_id) REFERENCES admissions(admission_id),
CONSTRAINT fk_med_doctor FOREIGN KEY (prescribed_by) REFERENCES doctors(doctor_id)
);

```

```

);

CREATE TABLE IF NOT EXISTS lab_tests (
test_id INT AUTO_INCREMENT PRIMARY KEY,
admission_id INT NOT NULL,
test_name VARCHAR(150) NOT NULL,
test_date DATE,
result VARCHAR(255),
result_status ENUM('Normal','Abnormal','Critical','Pending') DEFAULT 'Pending',
cost DECIMAL(8,2) DEFAULT 0,
CONSTRAINT fk_lab_admission FOREIGN KEY (admission_id) REFERENCES admissions(admission_id)
);

CREATE TABLE IF NOT EXISTS staff (
staff_id INT AUTO_INCREMENT PRIMARY KEY,
first_name VARCHAR(50) NOT NULL,
last_name VARCHAR(50) NOT NULL,
role ENUM('Nurse','Technician','Admin','Pharmacist','Ward Boy') NOT NULL,
department_id INT,
shift ENUM('Morning','Evening','Night'),
joining_date DATE,
salary DECIMAL(10,2),
CONSTRAINT fk_staff_dept FOREIGN KEY (department_id) REFERENCES departments(department_id)
);

-- =====
-- Hospital Dashboard Summary VIEW
-- Powers the live KPI cards on the React dashboard
-- =====
CREATE OR REPLACE VIEW vw_hospital_dashboard AS
SELECT
(SELECT COUNT(*) FROM patients) AS total_patients,
(SELECT COUNT(*) FROM admissions) AS total_admissions,
(SELECT COUNT(*) FROM admissions WHERE status='Admitted') AS active_admissions,
(SELECT COUNT(*) FROM doctors) AS total_doctors,
(SELECT COUNT(*) FROM staff) AS total_support_staff,
(SELECT ROUND(SUM(total_amount),0) FROM billing) AS gross_revenue,
(SELECT ROUND(SUM(amount_paid),0) FROM billing) AS revenue_collected,
(SELECT ROUND(AVG(DATEDIFF(
COALESCE(discharge_date, CURDATE()), admission_date
)),1) FROM admissions) AS avg_length_of_stay,
(SELECT ROUND(SUM(CASE WHEN severity='Critical' THEN 1 ELSE 0 END)
* 100.0 / COUNT(*), 1)
FROM admissions) AS critical_case_pct;

```

4. Backend — Flask REST API

The backend is a Flask application organized into a main entry point (app.py), a database connection module, and 8 route modules (one per resource, registered as Flask Blueprints).

Folder Structure

Folder structure

```
backend/
|-- app.py <- Flask entry point, registers all blueprints
|-- requirements.txt <- Flask, flask-cors, mysql-connector-python
|-- config/
| `-- db.py <- MySQL connection pool + query helpers
`-- routes/
    |-- patients.py <- CRUD
    |-- doctors.py <- CRUD
    |-- departments.py <- CRUD
    |-- admissions.py <- CRUD (with status filter)
    |-- billing.py <- CRUD
    |-- staff.py <- CRUD
    |-- lab_tests.py <- CRUD
    `-- analytics.py <- 18 read-only KPI/chart endpoints
```

app.py — Application Entry Point

backend/app.py

```
"""
Hospital Data Analysis – Flask Backend
Entry point: app.py
Run: python app.py
"""

from flask import Flask, jsonify
from flask_cors import CORS

from config.db import get_connection
from routes.patients import patients_bp
from routes.doctors import doctors_bp
from routes.departments import departments_bp
from routes.admissions import admissions_bp
from routes.billing import billing_bp
from routes.staff import staff_bp
from routes.lab_tests import lab_tests_bp
from routes.analytics import analytics_bp

app = Flask(__name__)
CORS(app) # allow React (localhost:3000) to call this API

# Register all blueprints
app.register_blueprint(patients_bp, url_prefix="/api/patients")
app.register_blueprint(doctors_bp, url_prefix="/api/doctors")
app.register_blueprint(departments_bp, url_prefix="/api/departments")
app.register_blueprint(admissions_bp, url_prefix="/api/admissions")
app.register_blueprint(billing_bp, url_prefix="/api/billing")
app.register_blueprint(staff_bp, url_prefix="/api/staff")
app.register_blueprint(lab_tests_bp, url_prefix="/api/lab-tests")
app.register_blueprint(analytics_bp, url_prefix="/api/analytics")

@app.route("/api/health")
def health_check():
```



```

"""Simple endpoint to verify backend + DB connection are alive."""
try:
    conn = get_connection()
    conn.close()
    return jsonify({"status": "ok", "database": "connected"})
except Exception as e:
    return jsonify({"status": "error", "message": str(e)}), 500

@app.route("/")
def index():
    return jsonify({
        "message": "Hospital Data Analysis API",
        "version": "1.0",
        "endpoints": [
            "/api/patients", "/api/doctors", "/api/departments",
            "/api/admissions", "/api/billing", "/api/staff",
            "/api/lab-tests", "/api/analytics"
        ]
    })

if __name__ == "__main__":
    app.run(debug=False, port=5000)

```

config/db.py — Database Connection Pool

backend/config/db.py

```

"""
MySQL connection configuration.
Edit the credentials below to match your local MySQL setup.
"""
import mysql.connector
from mysql.connector import pooling

DB_CONFIG = {
    "host": "localhost",
    "user": "root",
    "password": "YOUR_MYSQL_PASSWORD", # <-- change this to your MySQL password
    "database": "hospital_db",
    "port": 3306,
}

# Connection pool for better performance under multiple requests
_pool = pooling.MySQLConnectionPool(
    pool_name="hospital_pool",
    pool_size=5,
    **DB_CONFIG
)

def get_connection():
    """Returns a pooled MySQL connection. Caller is responsible for closing it."""
    return _pool.get_connection()

def run_query(query, params=None, fetch=True):
    """
    Helper to run a SELECT query and return list of dict rows.
    """
    conn = get_connection()
    cursor = conn.cursor(dictionary=True)
    cursor.execute(query, params or ())
    result = cursor.fetchall() if fetch else None

```

```

cursor.close()
conn.close()
return result

def run_write(query, params=None):
    """
    Helper to run INSERT / UPDATE / DELETE. Returns lastrowid (for INSERT).
    """
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute(query, params or ())
    conn.commit()
    last_id = cursor.lastrowid
    cursor.close()
    conn.close()
    return last_id

```

routes/admissions.py — Sample CRUD Module

Demonstrates the CRUD pattern used across all 7 resource modules, with JOINS across patients, doctors, and departments, plus an optional status filter.

backend/routes/admissions.py

```

"""
Admissions CRUD API
"""
from flask import Blueprint, request, jsonify
from config.db import run_query, run_write

admissions_bp = Blueprint("admissions", __name__)

@admissions_bp.route("/", methods=["GET"])
def get_admissions():
    status_filter = request.args.get("status")
    base_query = """
    SELECT a.*,
    CONCAT(p.first_name, ' ', p.last_name) AS patient_name,
    CONCAT(doc.first_name, ' ', doc.last_name) AS doctor_name,
    dept.department_name,
    DATEDIFF(COALESCE(a.discharge_date, CURDATE()), a.admission_date) AS los_days
    FROM admissions a
    JOIN patients p ON a.patient_id = p.patient_id
    JOIN doctors doc ON a.doctor_id = doc.doctor_id
    JOIN departments dept ON a.department_id = dept.department_id
    """
    if status_filter:
        base_query += " WHERE a.status = %s"
        rows = run_query(base_query + " ORDER BY a.admission_date DESC", (status_filter,))
    else:
        rows = run_query(base_query + " ORDER BY a.admission_date DESC")
    return jsonify(rows)

@admissions_bp.route("/<int:admission_id>", methods=["GET"])
def get_admission(admission_id):
    rows = run_query("""
    SELECT a.*,
    CONCAT(p.first_name, ' ', p.last_name) AS patient_name,
    CONCAT(doc.first_name, ' ', doc.last_name) AS doctor_name,
    dept.department_name
    FROM admissions a
    JOIN patients p ON a.patient_id = p.patient_id
    """

```

```

JOIN doctors doc ON a.doctor_id = doc.doctor_id
JOIN departments dept ON a.department_id = dept.department_id
WHERE a.admission_id = %s
""" , (admission_id,))
if not rows:
return jsonify({"error": "Admission not found"}), 404
return jsonify(rows[0])

@admissions_bp.route("/", methods=["POST"])
def create_admission():
data = request.json
new_id = run_write("""
INSERT INTO admissions
(patient_id, doctor_id, department_id, admission_date, discharge_date,
diagnosis, severity, admission_type, room_number, status)
VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s)
""" , (
data["patient_id"], data["doctor_id"], data["department_id"],
data["admission_date"], data.get("discharge_date"),

...(truncated for brevity – full file in repository)...

```

routes/analytics.py — Dashboard & KPI Endpoints (excerpt)

This module exposes the original SQL project's analytical queries — including window functions — as live JSON API endpoints.

```
backend/routes/analytics.py
```

[illegible]

```
@analytics_bp.route("/monthly-admissions", methods=["GET"])
def monthly_admissions():
    query = """
    SELECT YEAR(admission_date) AS yr, MONTH(admission_date) AS mo,
    MONTHNAME(admission_date) AS month_name, COUNT(*) AS admissions
    FROM admissions
    GROUP BY yr, mo, month_name
    ORDER BY yr, mo
    """
    return jsonify(run_query(query))

@analytics_bp.route("/severity-breakdown", methods=["GET"])
def severity_breakdown():
    query = """
    SELECT severity, COUNT(*) AS cases,
    ROUND(AVG(DATEDIFF(COALESCE(discharge_date, CURDATE()), admission_date)), 1) AS avg_los
    FROM admissions
    GROUP BY severity
    ORDER BY FIELD(severity, 'Mild', 'Moderate', 'Severe', 'Critical')
    """
    return jsonify(run_query(query))

...(truncated for brevity – full file in repository)...
```

5. Frontend — React Application

The frontend is a React 19 single-page application built with Vite, using React Router for navigation, Axios for API calls, and Recharts for data visualization. Styling is hand-written CSS using a custom design system (no UI framework).

Folder Structure

Folder structure

```
frontend/
|-- index.html
|-- vite.config.js <- Dev server + API proxy to Flask (port 5000)
`-- src/
  |-- main.jsx <- React entry point
  |-- App.jsx <- Routes definition
  |-- index.css <- Design system (CSS variables, components)
  |-- api/
  | `-- client.js <- Axios client + resource API helpers
  |-- components/
  | |-- Sidebar.jsx <- Left navigation
  | |-- Modal.jsx <- Reusable modal dialog
  | |-- Badge.jsx <- Severity / payment status badges
  | |-- Icons.jsx <- Inline SVG icon set
  | `-- VitalsLine.jsx <- Signature ECG-style divider motif
  `-- pages/
    |-- Dashboard.jsx <- KPIs + 6 charts
    |-- PatientsPage.jsx <- CRUD + search
    |-- DoctorsPage.jsx <- CRUD
    |-- DepartmentsPage.jsx <- CRUD
    |-- AdmissionsPage.jsx <- CRUD + status filter chips
    |-- BillingPage.jsx <- CRUD
    |-- LabTestsPage.jsx <- CRUD
    `-- StaffPage.jsx <- CRUD
```

App.jsx — Routing

```
frontend/src/App.jsx
```

```
import { Routes, Route } from 'react-router-dom'
import Sidebar from './components/Sidebar'
import Dashboard from './pages/Dashboard'
import PatientsPage from './pages/PatientsPage'
import DoctorsPage from './pages/DoctorsPage'
import DepartmentsPage from './pages/DepartmentsPage'
import AdmissionsPage from './pages/AdmissionsPage'
import BillingPage from './pages/BillingPage'
import LabTestsPage from './pages/LabTestsPage'
import StaffPage from './pages/StaffPage'

export default function App() {
  return (
    <div className="app-shell">
      <Sidebar />
      <main className="main-content">
        <Routes>
          <Route path="/" element={<Dashboard />} />
          <Route path="/patients" element={<PatientsPage />} />
          <Route path="/doctors" element={<DoctorsPage />} />
          <Route path="/departments" element={<DepartmentsPage />} />
          <Route path="/admissions" element={<AdmissionsPage />} />
          <Route path="/billing" element={<BillingPage />} />
          <Route path="/lab-tests" element={<LabTestsPage />} />
          <Route path="/staff" element={<StaffPage />} />
        </Routes>
      </main>
    </div>
  )
}
```

api/client.js — Axios API Client (excerpt)

```
frontend/src/api/client.js
```

[illegible]

```

create: (data) => api.post('/departments/', data).then(r => r.data),
update: (id, data) => api.put(`/departments/${id}`, data).then(r => r.data),
remove: (id) => api.delete(`/departments/${id}`).then(r => r.data),
}

export const Admissions = {
list: (status = '') => api.get(`/admissions/${status ? `?status=${status}` : ''}`).then(r => r.data),
get: (id) => api.get(`/admissions/${id}`).then(r => r.data),
create: (data) => api.post('/admissions/', data).then(r => r.data),
update: (id, data) => api.put(`/admissions/${id}`, data).then(r => r.data),
remove: (id) => api.delete(`/admissions/${id}`).then(r => r.data),
}

export const Billing = {
list: () => api.get('/billing/').then(r => r.data),
create: (data) => api.post('/billing/', data).then(r => r.data),
update: (id, data) => api.put(`/billing/${id}`, data).then(r => r.data),
remove: (id) => api.delete(`/billing/${id}`).then(r => r.data),
}

export const Staff = {
list: () => api.get('/staff/').then(r => r.data),
create: (data) => api.post('/staff/', data).then(r => r.data),
update: (id, data) => api.put(`/staff/${id}`, data).then(r => r.data),
remove: (id) => api.delete(`/staff/${id}`).then(r => r.data),
}

export const LabTests = {

...(truncated for brevity – full file in repository)...

```


components/Sidebar.jsx — Navigation

frontend/src/components/Sidebar.jsx

```
import { NavLink } from 'react-router-dom'
import {
  IconDashboard, IconPatients, IconDoctor, IconDept,
  IconAdmission, IconBilling, IconLab, IconStaff, IconPulse
} from './Icons'

const NAV_ITEMS = [
  { to: '/', label: 'Dashboard', icon: IconDashboard },
  { to: '/patients', label: 'Patients', icon: IconPatients },
  { to: '/doctors', label: 'Doctors', icon: IconDoctor },
  { to: '/departments', label: 'Departments', icon: IconDept },
  { to: '/admissions', label: 'Admissions', icon: IconAdmission },
  { to: '/billing', label: 'Billing', icon: IconBilling },
  { to: '/lab-tests', label: 'Lab Tests', icon: IconLab },
  { to: '/staff', label: 'Staff', icon: IconStaff },
]

export default function Sidebar() {
  return (
    <aside className="sidebar">
      <div className="sidebar-brand">
        <IconPulse className="pulse-icon" style={{ color: '#02C39A' }} />
        <span className="sidebar-brand-text">Medi<span>Track</span></span></div>

        <nav className="sidebar-nav">
          {NAV_ITEMS.map(({ to, label, icon: Icon }) => (
            <NavLink
              key={to}
              to={to}
              end={to === '/'}
              className={({ isActive }) => `sidebar-link${isActive ? ' active' : ''}`}
            >
              <Icon className="icon" />
              {label}
            </NavLink>
          ))}
        </nav>

        <div className="sidebar-footer">
          hospital_db · MySQL 8.0
        </div>
      </aside>
    )
  }
}
```

pages/Dashboard.jsx (excerpt)

Fetches 8 analytics endpoints in parallel and renders KPI cards + 6 Recharts visualizations.

frontend/src/pages/Dashboard.jsx

```
import { useEffect, useState } from 'react'
import {
  ResponsiveContainer, BarChart, Bar, XAxis, YAxis, CartesianGrid, Tooltip,
  LineChart, Line, PieChart, Pie, Cell, Legend,
} from 'recharts'
import { Analytics, checkHealth } from '../api/client'
import VitalsLine from '../components/VitalsLine'

const COLORS = ['#028090', '#1C7293', '#065A82', '#02C39A', '#5C6F7C', '#0A2342']
```

```

function fmtMoney(n) {
  if (n == null) return '■0'
  const v = Number(n)
  if (v >= 100000) return `■${(v / 100000).toFixed(1)}L`
  if (v >= 1000) return `■${(v / 1000).toFixed(1)}K`
  return `■${v.toFixed(0)}`
}

export default function Dashboard() {
  const [summary, setSummary] = useState(null)
  const [gender, setGender] = useState([])
  const [monthlyAdm, setMonthlyAdm] = useState([])
  const [severity, setSeverity] = useState([])
  const [deptPerf, setDeptPerf] = useState([])
  const [costBreakdown, setCostBreakdown] = useState(null)
  const [topDiagnoses, setTopDiagnoses] = useState([])
  const [dbStatus, setDbStatus] = useState(null)
  const [loading, setLoading] = useState(true)

  useEffect(() => {
    Promise.allSettled([
      checkHealth(),
      Analytics.dashboardSummary(),
      Analytics.genderDistribution(),
      Analytics.monthlyAdmissions(),
      Analytics.severityBreakdown(),
      Analytics.departmentPerformance(),
      Analytics.costBreakdown(),
      Analytics.topDiagnoses(),
    ]).then(([health, sum, gen, ma, sev, dept, cost, diag]) => {
      setDbStatus(health.status === 'fulfilled' ? health.value : { status: 'error' })
      if (sum.status === 'fulfilled') setSummary(sum.value)
      if (gen.status === 'fulfilled') setGender(gen.value)
      if (ma.status === 'fulfilled') setMonthlyAdm(ma.value)
      if (sev.status === 'fulfilled') setSeverity(sev.value)
      if (dept.status === 'fulfilled') setDeptPerf(dept.value.slice(0, 6))
      if (cost.status === 'fulfilled') setCostBreakdown(cost.value)
      if (diag.status === 'fulfilled') setTopDiagnoses(diag.value.slice(0, 6))
      setLoading(false)
    })
  }, [])

  const costData = costBreakdown ? [
    { name: 'Surgery', value: Number(costBreakdown.surgery_cost || 0) },
    { name: 'Medication', value: Number(costBreakdown.medication_cost || 0) },
    { name: 'Room', value: Number(costBreakdown.room_charges || 0) },
    { name: 'Consultation', value: Number(costBreakdown.consultation_fee || 0) },
  ] : []

  return (
    <div>
      <div className="page-header">
        <div>
          <div className="page-eyebrow">Overview</div>
          <h1>Hospital Dashboard</h1>
          <div className="page-subtitle">Live KPIs pulled directly from MySQL via Flask API</div>
        </div>
      </div>

      {dbStatus && (
        ...(truncated for brevity – full file in repository)...
      )}
    </div>
  )
}

```

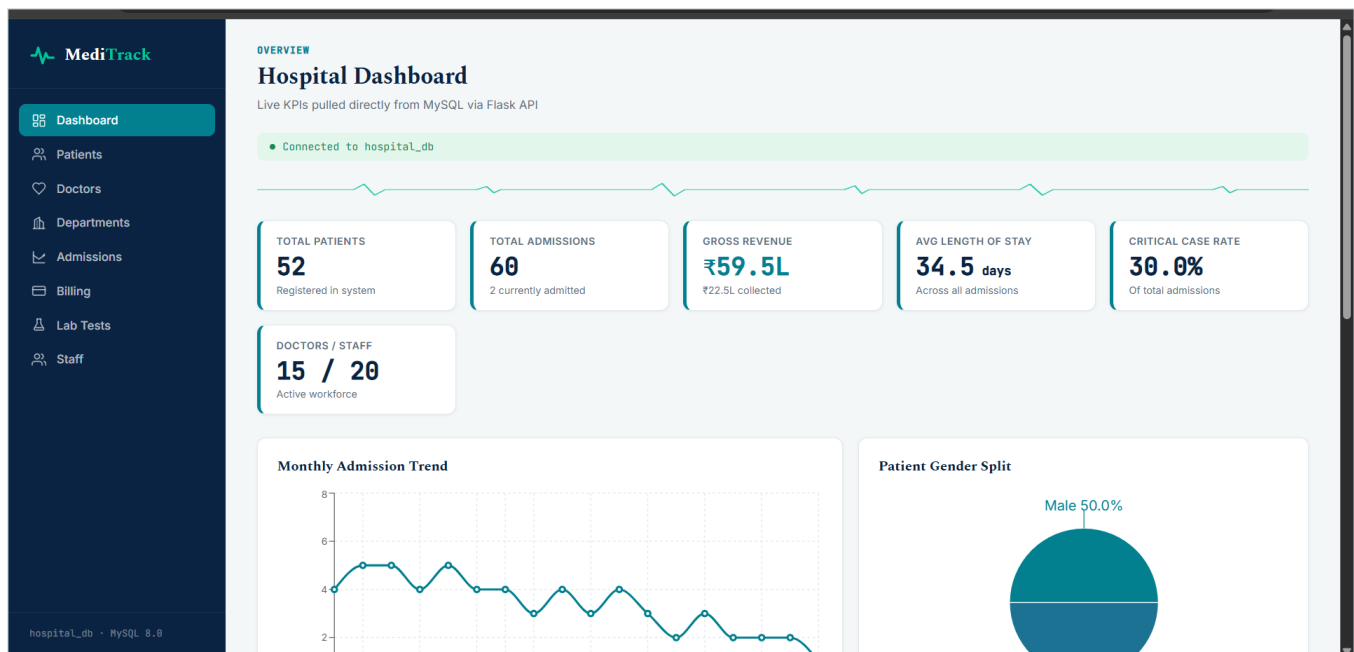
6. Live Application — Page-by-Page Walkthrough

The following screenshots were captured from the running application, connected live to the hospital_db MySQL database.

6.1 Dashboard

Route: `/` | API: `GET /api/analytics/*` (8 endpoints)

The landing page. Shows 6 KPI cards (Total Patients, Total Admissions, Gross Revenue, Avg Length of Stay, Critical Case Rate, Doctors/Staff) pulled live from the `vw_hospital_dashboard` SQL view, plus a monthly admission trend line chart and a patient gender split pie chart, both rendered with Recharts from live query results.



6.2 Patients

Route: /patients | API: GET / POST / PUT / DELETE /api/patients/

Full CRUD page for the patients table. Includes a live search bar (filters by name or phone via a SQL LIKE query) and Edit/Delete actions per row. The 'Add Patient' button opens a modal form that POSTs directly to the Flask API.

MediTrack

Dashboard

Patients

Doctors

Departments

Admissions

Billing

Lab Tests

Staff

hospital_db • MySQL 8.0

RECORDS

Patients

52 patients registered

+ Add Patient

Q Search by name or phone...

ID	NAME	DOB	GENDER	BLOOD GROUP	PHONE	EMAIL	
#60	Rakesh Pawar	Sat, 22 No	Male	O+	9900001060	rakesh.p@email.com	
#59	Anjali Bhosale	Sat, 14 Ma	Female	B+	9900001059	anjali.b@email.com	
#50	Varsha Dalvi	Sat, 09 Ju	Female	AB-	9900001050	varsha.d@email.com	
#49	Prakash Hegde	Mon, 16 Ma	Male	O+	9900001049	prakash.h@email.com	
#48	Leena Sawant	Sat, 28 Oc	Female	B-	9900001048	leena.s@email.com	
#47	Santosh More	Mon, 11 Ju	Male	A+	9900001047	santosh.m@email.com	
#46	Bhavna Parikh	Sat, 03 Fe	Female	O+	9900001046	bhavna.p@email.com	
#45	Girish Kamat	Thu, 22 Au	Male	AB+	9900001045	girish.k@email.com	
#44	Poornima Naik	Thu, 14 Ma	Female	B+	9900001044	poornima.n@email.com	
#43	Umesh Chandra	Wed, 31 Ja	Male	O+	9900001043	umesh.c@email.com	

6.3 Doctors

Route: /doctors | API: GET / POST / PUT / DELETE /api/doctors/

Lists all doctors with their specialization, assigned department (via SQL JOIN), and years of experience. Supports full CRUD with a department dropdown populated from the departments table.

MediTrack

Dashboard

Patients

Doctors

Departments

Admissions

Billing

Lab Tests

Staff

hospital_db · MySQL 8.0

RECORDS

Doctors

15 doctors on staff

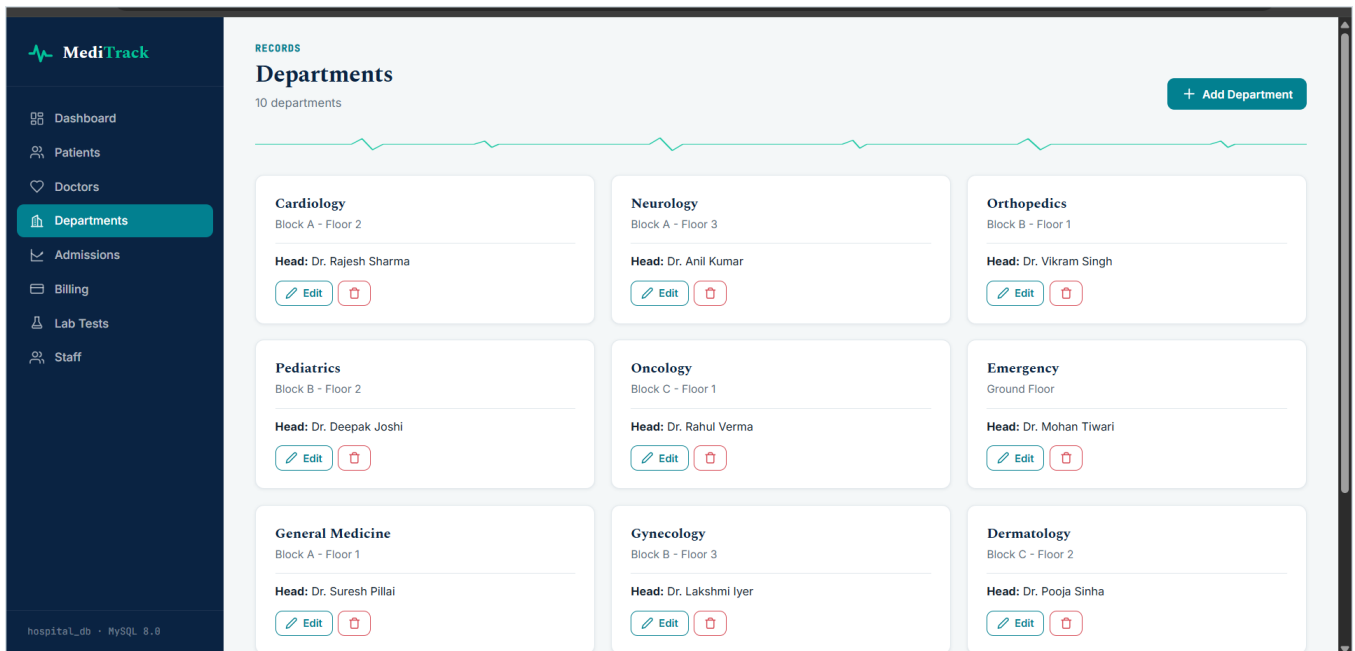
+ Add Doctor

ID	NAME	SPECIALIZATION	DEPARTMENT	EXPERIENCE	PHONE		
#1	Dr. Rajesh Sharma	Cardiologist	Cardiology	15 yrs	9810001001		
#2	Dr. Priya Mehta	Cardiologist	Cardiology	8 yrs	9810001002		
#3	Dr. Anil Kumar	Neurologist	Neurology	20 yrs	9810001003		
#4	Dr. Sneha Patel	Neurologist	Neurology	10 yrs	9810001004		
#5	Dr. Vikram Singh	Orthopedic Surgeon	Orthopedics	18 yrs	9810001005		
#6	Dr. Kavita Rao	Orthopedic Surgeon	Orthopedics	6 yrs	9810001006		
#7	Dr. Deepak Joshi	Pediatrician	Pediatrics	12 yrs	9810001007		
#8	Dr. Sunita Gupta	Pediatrician	Pediatrics	9 yrs	9810001008		
#9	Dr. Rahul Verma	Oncologist	Oncology	22 yrs	9810001009		
#10	Dr. Anita Desai	Oncologist	Oncology	14 yrs	9810001010		
#11	Dr. Mohan Tiwari	Emergency Medicine	Emergency	11 yrs	9810001011		

6.4 Departments

Route: /departments | API: GET / POST / PUT / DELETE /api/departments/

Card-based layout showing all 10 hospital departments, their physical location, and head doctor (resolved via a LEFT JOIN to doctors). Each card has inline Edit and Delete actions.



6.5 Admissions

Route: /admissions | API: GET / POST / PUT / DELETE /api/admissions/

The most complex CRUD page — joins patients, doctors, and departments tables, computes length-of-stay (LOS) using DATEDIFF, and color-codes severity with badges. Filter chips (All / Admitted / Discharged / ICU) call the same endpoint with a status query parameter.

MediTrack

Dashboard

Patients

Doctors

Departments

Admissions

Billing

Lab Tests

Staff

hospital_db • MySQL 8.0

RECORDS

Admissions

52 admission records

+ New Admission

AllAdmittedDischargedICU

ID	PATIENT	DOCTOR	DEPARTMENT	DIAGNOSIS	SEVERITY	ADMITTED	LOS	STATUS		
#60	Rakesh Pawar	Dr. Lakshmi Iyer	Gynecology	High-Risk Pregnancy Monitoring	SEVERE	Sat, 01 Ju	754 days	Admitted		
#59	Anjali Bhosale	Dr. Rahul Verma	Oncology	Leukemia Treatment Ongoing	CRITICAL	Fri, 10 Ma	776 days	Admitted		
#50	Varsha Dalvi	Dr. Vikram Singh	Orthopedics	Spinal Fusion Surgery	SEVERE	Fri, 12 Ja	8 days	Discharged		
#49	Prakash Hegde	Dr. Rajesh Sharma	Cardiology	Coronary Bypass Surgery	CRITICAL	Mon, 08 Ja	9 days	Discharged		
#48	Leena Sawant	Dr. Suresh Pillai	General Medicine	Malaria Treatment	MODERATE	Sun, 10 De	8 days	Discharged		
#47	Santosh More	Dr. Mohan Tiwari	Emergency	Cardiac Arrest - Stabilized	CRITICAL	Tue, 05 De	1 days	Discharged		
#46	Bhavna Parikh	Dr. Anil Kumar	Neurology	Multiple Sclerosis Evaluation	SEVERE	Fri, 01 De	9 days	Discharged		
#45	Girish Kamat	Dr. Lakshmi Iyer	Gynecology	Ectopic Pregnancy	CRITICAL	Mon, 20 No	4 days	Discharged		
#44	Poornima Naik	Dr. Deepak Joshi	Pediatrics	Chicken Pox (Pediatric)	MILD	Fri, 10 No	5 days	Discharged		
#43	Umesh Chandra	Dr. Rahul Verma	Oncology	Liver Cancer Chemotherapy	CRITICAL	Sun, 05 No	35 days	Discharged		

6.6 Billing

Route: /billing | API: GET / POST / PUT / DELETE /api/billing/

Displays every bill with total amount, insurance coverage, amount paid, and payment status (Paid / Partial / Pending) as color-coded badges. The header summarizes total billed and collected amounts across all records.

MediTrack

Dashboard

Patients

Doctors

Departments

Admissions

Billing

Lab Tests

Staff

hospital_db • MySQL 8.0

FINANCE

Billing

50 bills • ₹50,98,000 billed • ₹19,50,000 collected

+ New Bill

BILL ID	PATIENT	DIAGNOSIS	TOTAL	INSURANCE	PAID	STATUS	METHOD		
#50	Varsha Dalvi	Spinal Fusion Surgery	₹1,95,000	₹1,20,000	₹75,000	PARTIAL	Insurance		
#49	Prakash Hegde	Coronary Bypass Surgery	₹3,10,000	₹2,20,000	₹90,000	PARTIAL	Insurance		
#48	Leena Sawant	Malaria Treatment	₹26,000	₹12,000	₹14,000	PAID	UPI		
#46	Bhavna Parikh	Multiple Sclerosis Evaluation	₹74,000	₹50,000	₹24,000	PARTIAL	Insurance		
#43	Umesh Chandra	Liver Cancer Chemotherapy	₹4,20,000	₹3,00,000	₹1,20,000	PARTIAL	Insurance		
#47	Santosh More	Cardiac Arrest - Stabilized	₹30,000	₹0	₹30,000	PAID	Card		
#45	Girish Kamat	Ectopic Pregnancy	₹68,000	₹40,000	₹28,000	PARTIAL	Insurance		
#44	Poornima Naik	Chicken Pox (Pediatric)	₹12,000	₹5,000	₹7,000	PAID	Cash		
#42	Nidhi Rastogi	Knee Arthroscopy	₹42,000	₹25,000	₹17,000	PAID	Card		
#41	Alok Bajpai	Valve Replacement Surgery	₹2,80,000	₹2,00,000	₹80,000	PARTIAL	Insurance		
#40	Vandana Sethi	Cholera Treatment	₹22,000	₹10,000	₹12,000	PAID	UPI		

6.7 Lab Tests

Route: /lab-tests | API: GET / POST / PUT / DELETE /api/lab-tests/

Shows diagnostic test records joined to their parent admission and patient, with result status badges (Normal / Abnormal / Critical) and per-test cost.

MediTrack

Dashboard

Patients

Doctors

Departments

Admissions

Billing

Lab Tests

Staff

hospital_db · MySQL 8.0

DIAGNOSTICS

Lab Tests

26 tests recorded

+ Add Lab Test

ID	PATIENT	TEST NAME	DATE	RESULT	STATUS	COST		
#26	Prakash Hegde	Coronary CTA	Mon, 08 Ja	90% LAD occlusion	CRITICAL	₹7,000		
#25	Santosh More	ABG	Tue, 05 De	pH 7.1 - Acidosis	CRITICAL	₹1,000		
#23	Umesh Chandra	AFP Marker	Sun, 05 No	1200 IU/mL (High)	CRITICAL	₹1,200		
#24	Umesh Chandra	MRI Abdomen	Sun, 05 No	Hepatocellular carcinoma	CRITICAL	₹5,000		
#22	Alok Bajpai	Cardiac Catheterization	Tue, 10 Oc	Severe mitral regurgitation	CRITICAL	₹9,000		
#21	Reena Gaur	Lumbar Puncture	Tue, 05 Se	Bacterial Meningitis	CRITICAL	₹2,500		
#20	Hemant Soni	PSA	Fri, 01 Se	45 ng/mL (High)	CRITICAL	₹900		
#19	Geeta Sharma	CBC	Wed, 02 Au	Platelet 40000 (Low)	CRITICAL	₹500		
#18	Geeta Sharma	NS1 Antigen	Tue, 01 Au	Dengue Positive	ABNORMAL	₹800		
#16	Madhuri Patil	Colonoscopy	Wed, 12 Ju	Polyp in sigmoid colon	ABNORMAL	₹4,000		
#17	Madhuri Patil	CEA Marker	Wed, 12 Ju	28 ng/mL (High)	ABNORMAL	₹1,000		

6.8 Staff

Route: /staff | API: GET / POST / PUT / DELETE /api/staff/

Workforce management page — lists nurses, technicians, pharmacists, admin staff, and ward boys with their department, shift (Morning/Evening/Night), and salary.

MediTrack

Dashboard

Patients

Doctors

Departments

Admissions

Billing

Lab Tests

Staff

hospital_db · MySQL 8.0

WORKFORCE

Staff

20 support staff

+ Add Staff

ID	NAME	ROLE	DEPARTMENT	SHIFT	SALARY	
#1	Sunanda Pillai	Nurse	Cardiology	Morning	₹42,000	
#2	Ramesh Nair	Nurse	Neurology	Evening	₹40,000	
#3	Deepa Varma	Nurse	Orthopedics	Night	₹39,000	
#4	Asha Krishnan	Nurse	Pediatrics	Morning	₹43,000	
#5	Monika Shah	Nurse	Oncology	Evening	₹38,000	
#6	Rajan Mehta	Technician	Cardiology	Morning	₹35,000	
#7	Pradeep Jain	Technician	Neurology	Evening	₹34,000	
#8	Sonal Bhatt	Technician	Emergency	Night	₹33,000	
#9	Kavita Desai	Admin	General Medicine	Morning	₹30,000	
#10	Prakash Soni	Admin	Emergency	Morning	₹32,000	
#11	Meenakshi Pandey	Pharmacist	General Medicine	Morning	₹36,000	

7. API Endpoint Reference

CRUD Pattern (identical across all 7 resources)

CRUD endpoint pattern

```
GET /api/<resource>/ -> list all records
GET /api/<resource>/<id> -> get single record
POST /api/<resource>/ -> create new record
PUT /api/<resource>/<id> -> update existing record
DELETE /api/<resource>/<id> -> delete record
```

Resources: patients, doctors, departments, admissions, billing, staff, lab-tests

Analytics Endpoints (read-only, power the dashboard)

18 analytics endpoints

```
GET /api/analytics/dashboard-summary (SQL VIEW: vw_hospital_dashboard)
GET /api/analytics/gender-distribution
GET /api/analytics/age-distribution
GET /api/analytics/admission-summary
GET /api/analytics/monthly-admissions
GET /api/analytics/severity-breakdown
GET /api/analytics/department-performance
GET /api/analytics/doctor-performance
GET /api/analytics/revenue-summary
GET /api/analytics/cost-breakdown
GET /api/analytics/payment-methods
GET /api/analytics/monthly-revenue
GET /api/analytics/top-diagnoses
GET /api/analytics/lab-test-summary
GET /api/analytics/staff-summary
GET /api/analytics/revenue-running-total (window function: SUM OVER)
GET /api/analytics/admission-growth (window function: LAG)
GET /api/analytics/doctor-rank-by-department (window function: RANK)
```

8. Setup & Run Instructions

Prerequisites

- MySQL 8.0+ with the hospital_db database already created (schema + seed data loaded)
- Python 3.10+ with pip
- Node.js 18+ with npm

1. Backend Setup

Backend commands

```
cd backend
pip install -r requirements.txt

# Edit config/db.py and set your MySQL password:
# "password": "YOUR_MYSQL_PASSWORD"

python app.py
# Server runs on http://127.0.0.1:5000
# Verify: open http://127.0.0.1:5000/api/health
```

2. Frontend Setup

Frontend commands

```
cd frontend
npm install
npm run dev
# Server runs on http://localhost:3000
# Vite proxies all /api/* calls to the Flask backend on port 5000
```

Note: both the backend and frontend dev servers must be running simultaneously (in two separate terminals) for the application to function.

9. Summary

- Converts a SQL-only data analysis portfolio project into a fully working full-stack web application
- Backend: Flask REST API with 7 CRUD modules and 18 analytics endpoints, using parameterized queries (SQL-injection safe) and a pooled MySQL connection
- Frontend: React 19 + Vite single-page app with 8 pages, live charts (Recharts), modals, search, and filtering — all backed by real-time API calls
- Every chart and KPI on the dashboard is the live output of a SQL query — including window functions (RANK, LAG, SUM OVER) — not static or mocked data
- Demonstrates the complete data pipeline: MySQL schema design → REST API → interactive UI

Database: `hospital_db` | Backend: `Flask` (port 5000) | Frontend: `React + Vite` (port 3000)

MediTrack — Hospital Data Analysis Full-Stack Project